

HELIOS

Updated Project Knowledge Base

An Alpha-Driven Quantitative Trading System with Fault-Tolerant and Low-Latency
Trading Infrastructure

This document supersedes the systems-first framing of the original HELIOS synopsis. It records the upgraded project direction: alpha-driven trading as the primary focus, with fault tolerance and low latency as the second and third priorities.

Note on status: This document reflects a new decision on project direction and priority ordering. It is not yet reflected in the previously submitted synopsis, which still describes systems-first sequencing. The quant strategy, alpha, risk engine, OMS, fault tolerance, and low-latency/colocation discussion are already supported by the existing synopsis — the alpha-first priority and the specific Sharpe / Fitness / Turnover thresholds below are the newly upgraded requirements.

1. What HELIOS Is

HELIOS is a **fully alpha-driven quantitative trading system**, designed around three major goals:

- **Alpha-driven quantitative trading** — the primary focus.
- **Fault-tolerant trading infrastructure** — the reliability layer.
- **Low-latency trading infrastructure** — the performance layer.

The project should be understood primarily as an **alpha research** → **simulation** → **validation** → **backtesting** → **P&L generation** → **execution system**, rather than simply as a distributed matching engine.

The overall philosophy is:

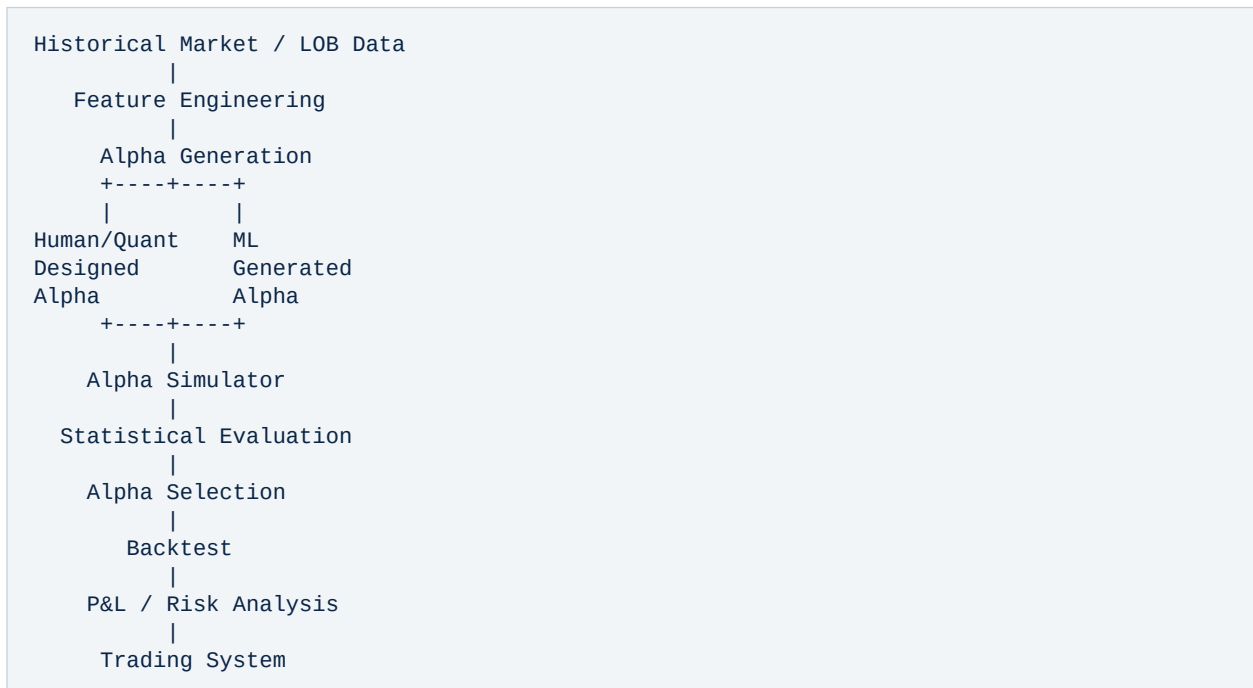
Generate an alpha → simulate it → evaluate whether it satisfies the required quantitative criteria → backtest it on historical market data → measure P&L and risk → retain promising alphas → feed them into the trading system.

Alpha can be:

- **Manually designed by us**, using quantitative / market-microstructure ideas, or
- **Generated / discovered with ML**, where ML searches for useful predictive representations and signals.

2. Primary Project Goal — Alpha-Driven Trading

The **Quant / Alpha component is now the highest priority** of HELIOS. The system should allow the team to develop multiple candidate alphas and systematically evaluate them.



The goal is **not simply to create a model that predicts price direction**. The goal is to produce an alpha that can survive quantitative evaluation and ultimately produce a **profitable trading strategy under**

realistic historical simulation.

3. Alpha Generation

HELIOS should support two major routes for generating alpha.

A. Quant-Designed Alpha

We can create alpha ourselves using market-microstructure / statistical intuition. Potential inputs include:

- Order-book imbalance
- Order-flow imbalance
- Spread
- Mid-price
- Microprice
- Volume
- Order-flow intensity
- Volatility
- Momentum
- Returns
- Rolling statistics
- Queue information
- LOB dynamics

```
LOB
|
Features
|
Mathematical / statistical combination
|
Alpha
```

B. ML-Generated Alpha

ML can be used to discover or learn predictive signals from market data.

```
LOB / Event Stream
|
ML Model
|
Learned representation
|
Prediction
|
Alpha
```

The ML component can include approaches such as:

- GNN
- Causal Transformer
- Self-supervised representation learning
- LOB models
- Transfer learning

The ML model should ultimately be evaluated based on whether its output can become a **useful trading alpha**, not merely whether it achieves good prediction accuracy.

4. Alpha Simulation

Every candidate alpha should first be **simulated / evaluated before being treated as a trading strategy**. The alpha pipeline should calculate:

- Expected return / predictive strength
- Sharpe ratio
- Fitness
- Turnover
- Drawdown
- Volatility
- Hit rate, where appropriate
- IC / correlation metrics where appropriate
- P&L
- Risk-adjusted performance
- Transaction-cost sensitivity
- Stability across time periods
- Robustness across market regimes

The initial target constraints are:

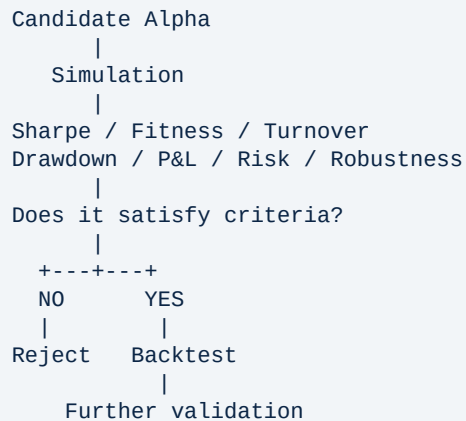
```
Sharpe   > 1
Fitness  > 1

Turnover :
  > 1%
  < 70%
```

These thresholds are **project selection criteria**, not guaranteed outcomes. An alpha should not be considered successful merely because it produces positive raw P&L — it should satisfy the project's major quantitative criteria and demonstrate robustness.

5. Alpha Selection

The system should automatically or systematically filter candidate alphas.



The project should maintain a record of candidate alphas and their evaluation results, allowing comparison across Alpha A, Alpha B, Alpha C, Alpha D, etc., to determine which signals are actually useful.

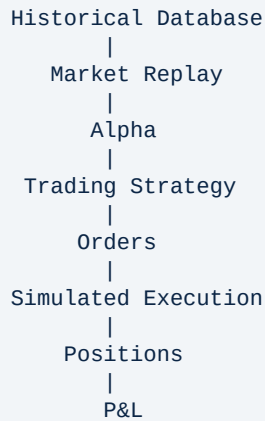
6. Backtesting

Promising alphas are then tested against historical market / order-book data. The backtest should simulate:

- Signal generation
- Order generation
- Execution
- Fills
- Position changes
- Transaction costs
- Slippage, where modelled
- Portfolio value
- P&L
- Risk

The goal is:

Can the alpha produce realistic positive P&L when applied to historical market conditions?



The backtesting system must avoid **look-ahead bias, data leakage, and unrealistic execution assumptions.**

7. Trading Strategy

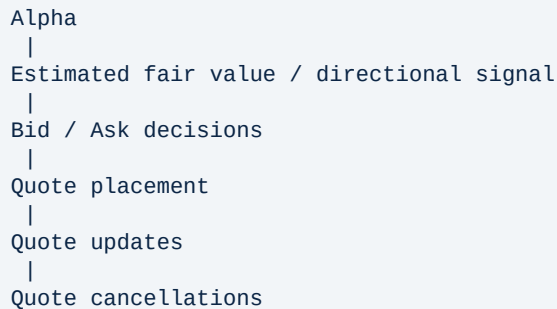
The alpha is not itself the complete trading strategy. The distinction is:

Alpha — “What does our model believe will happen?”

Strategy — “Given that belief, what orders should we place?”

The strategy layer converts alpha into actual trading decisions. The primary strategy direction is:

Alpha-driven HFT-style continuous two-sided quoting.



The strategy continuously reacts to changing market conditions rather than simply creating one buy order and one sell order. This is consistent with the revised HELIOS synopsis.

8. Risk Engine

The strategy does not have unlimited authority to trade. Every generated order goes through a separate risk layer.

```
Alpha
|
Strategy
|
Risk Engine
|
OMS
|
Execution
```

The risk engine checks:

- Position limits
- Exposure limits
- Maximum order size
- Price bands
- Abnormal order rate
- Kill switch
- Circuit breaker

The distinction must remain clear:

Strategy decides what we want to trade.

Risk decides whether we're allowed to trade it.

The revised synopsis explicitly defines this separation.

9. Paper Trading

The current practical implementation is **paper trading / simulated trading**, not real-money trading. Orders generated by the strategy should go through simulated execution.

```
Alpha
|
Strategy
|
Risk
|
Paper Order
|
Simulated Execution
|
Fill
|
Position
|
P&L
```

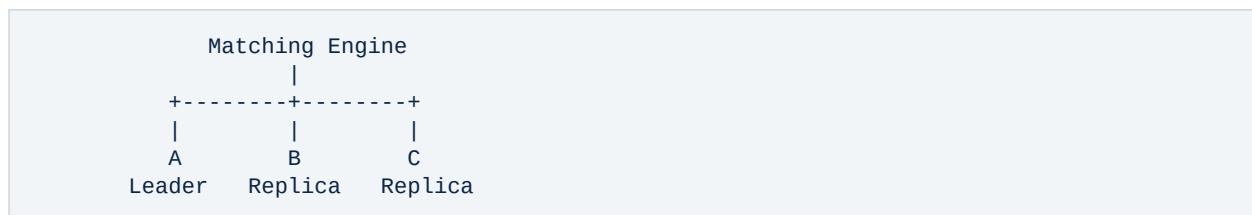
This allows the team to determine whether an alpha / strategy would have generated profit without connecting to a real exchange with real capital.

10. Fault-Tolerant System — Secondary Priority

The second major task is to make the trading infrastructure fault tolerant. The distributed systems component includes:

- Distributed matching engine
- Replication
- SDR
- Quorum
- Failure detection
- Rollback
- Deterministic replay
- Fault injection
- Chaos testing
- TLA+
- TLC
- Linearizability

The basic architecture is:



SDR means:

Speculative execution with deterministic rollback.

The leader can execute speculatively, replicate the operation, and commit after quorum agreement. If a conflict or failure occurs, the system can rollback and deterministically replay from the committed state / event history.

The matching engine must support **continuous, multi-level, price-time-priority matching, partial fills, and multiple counterparties**, rather than simple one-to-one matching.

Important priority change: the fault-tolerant distributed system is not the primary focus anymore. It is the second major task, after establishing the alpha-driven trading system.

11. Low-Latency System — Third Priority

The third major goal is to make the system low latency and HFT-oriented. The intended architecture is:

```
Exchange
|
Colocation
|
Low-Latency NIC
|
CPU-Pinned Thread
|
C++ Decoding / Parsing
|
LOB
|
Quant / Alpha
|
Strategy
```

The objective is to minimize unnecessary latency between market-data arrival, information extraction, signal generation, trading decision, and order transmission.

12. Colocation

In a production HFT environment, the server would ideally be colocated near the exchange infrastructure, reducing network distance and therefore latency:

```
Exchange
|
| very short network path
|
Trading Server
```

Physical exchange colocation is not available for the academic project. HELIOS should not claim actual colocation with a real exchange. Instead, the project builds and benchmarks a low-latency cloud-based environment that approximates the architectural principles as much as practical — consistent with the existing synopsis, which already treats physical colocation and similar production infrastructure as an extension rather than an implemented academic-prototype capability.

13. Low-Latency Cloud

Since physical colocation is unavailable, cloud infrastructure is used to build the closest practical approximation. The goal is to minimize:

- Network latency
- Jitter
- Scheduling delays
- Memory overhead
- Serialization overhead
- Unnecessary network hops
- CPU contention

Potential architecture:

```
Cloud Infrastructure
|
Dedicated / isolated CPU resources
|
CPU-pinned latency-sensitive thread
|
Low-overhead networking
|
C++ market-data processing
|
LOB
|
Alpha / Strategy
```

The project should **measure the latency rather than merely claiming that the system is low latency**. Important measurements: P50, P95, and P99 latency, throughput, jitter, processing latency, signal-generation latency, and order-generation latency.

14. C++ Role

C++ is intended for the latency-sensitive portions of the trading pipeline. Its responsibilities can include:

```
Network data
|
C++ decoding
|
Market-message parsing
|
LOB processing
|
Feature computation
|
Low-latency strategy/signal path
```

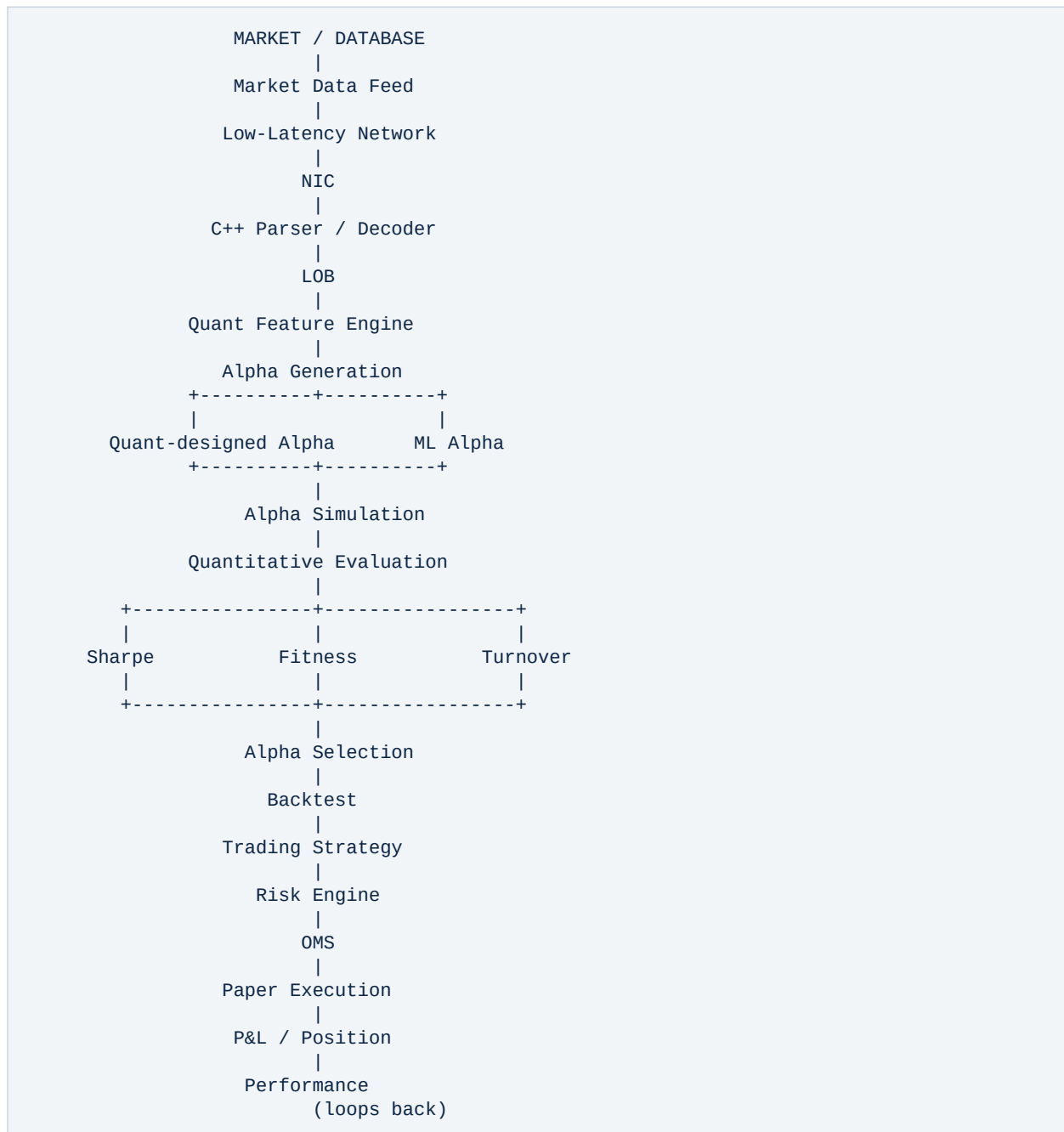
The NIC and C++ are not doing identical work.

NIC	Fast packet reception and potentially low-level packet processing.
C++	Application-level market-data interpretation, LOB processing, quantitative calculations, and latency-sensitive trading logic.

A Smart NIC / FPGA can optionally perform additional packet preprocessing, but **actual FPGA hardware is not currently part of the academic implementation**.

15. Complete Updated HELIOS Pipeline

The complete upgraded system should be understood as:



Then, once the strategy side works, the execution infrastructure can be upgraded:

```
Paper Execution
|
Fault-Tolerant Matching Engine
|
SDR
|
Replication
|
Fault Recovery
```

And the entire thing is optimized for low latency:

```
Colocation concept
|
Low-latency network
|
NIC
|
CPU pinning
|
C++
|
Fast LOB / Quant / Strategy
```

16. New Priority Order

This is the biggest change from the previous HELIOS understanding.

Priority 1 — Alpha / Quant (Primary project focus)

```
Generate Alpha
|
Simulate
|
Evaluate
|
Sharpe > 1
Fitness > 1
1% < Turnover < 70%
|
Backtest
|
P&L
|
Risk / robustness
```

Priority 2 — Fault Tolerance

```
Matching Engine
|
SDR
|
Replication
|
Failure Detection
|
Rollback / Replay
|
TLA+ / TLC
```

Priority 3 — Low Latency

```
Colocation architecture
|
Low-latency cloud alternative
|
NIC
|
CPU pinning
|
C++
|
Low-latency market-data -> signal -> order path
```

17. One-Sentence Definition of the Upgraded HELIOS

HELIOS is an alpha-driven quantitative trading system that generates and validates statistical/ML alphas, simulates and backtests them to measure risk-adjusted P&L, and ultimately executes those strategies through fault-tolerant and low-latency trading infrastructure.

This is now the working definition of HELIOS going forward.